Assignment 4: MIPS Game Project

Tic-Tac-Toe, Blackjack and Hangman in MIPS assembly

Dylan Flores, SDSU, Fall 2026, October 18, 2026

How to run the games

Each game is one file: tictactoe.asm, blackjack.asm and hangman.asm. They all use the same Bitmap Display settings. All of the code, the Java versions and the display images are also on my GitHub at github.com/dflo042/cs in the mips-game-project folder.

- Open the file in MARS 4.5 and assemble it (F3).
- Tools, then Bitmap Display. Unit Width 4, Unit Height 4, Display Width 256, Display Height 256, Base address 0x10008000 ($gp). Click Connect to MIPS.
- Run (F5). Prompts show up in the Run I/O box at the bottom and the game is drawn in the Bitmap Display window. The board or cards are also printed as text in the console.
- Type an answer and press Enter. Bad input gives an error message and asks again. N at the play again prompt quits.

Java versions: javac TicTacToe.java Blackjack.java Hangman.java, then java TicTacToe, java Blackjack or java Hangman. They take the same input and print a text version of the display after each move.

Controls: Tic-Tac-Toe wants a cell number 1 to 9. Blackjack wants H or S (either case). Hangman wants one letter. All three ask Y or N to play again. Anything else is rejected with a message and nothing changes.

How the display and input work

I wrote one small set of graphics and input procedures and put the same copy in each file so every file assembles on its own. The display is 64 by 64 units, one word per unit holding a color as 0x00RRGGBB, 4096 words starting at 0x10008000. A pixel's address is 0x10008000 + (y * 64 + x) * 4. draw_pixel does that with two shifts and an add and checks that x and y are inside 0 to 63 first so a bad coordinate can never overwrite anything else.

- fill_screen(color) clears the screen. draw_rect(x, y, w, h, color) is a filled rectangle. It has five arguments so the color is passed on the stack and the procedure reads it with lw $t9, 0($sp) before saving its own registers.
- draw_glyph(x, y, char, color) draws one character from a 3 by 5 font I made, stored as 5 bytes per character with bit 2 as the left pixel. It has 0-9, A-Z, space, _, ?, -, : and !. draw_text draws a string 4 pixels per character, so 15 fit across.
- read_line uses syscall 8 to read a whole line and strips the newline. parse_int (Tic-Tac-Toe) turns the text into a number and reports whether it really was one. I did this instead of syscall 5 because syscall 5 throws an error in MARS if you type letters. to_upper lets lower case letters work.

Conventions: $a0-$a3 are arguments, $v0/$v1 return values. $t registers are temporary and never expected to survive a jal (loops that only call draw_pixel keep a counter in $t8 since draw_pixel only uses $t0 and $t1). Every procedure that uses $s registers pushes them and $ra on the stack first and pops them at the end. Leaf procedures do not touch the stack. Game state is in .data as byte arrays and

word counters. .eqv is used for colors and sizes and a macro for the print syscalls.

Game 1: Tic-Tac-Toe

Two players take turns typing a cell number. The board is a 9 byte array (0 empty, 1 X, 2 O) and the eight winning lines are a 24 byte table of cell indexes, so check_win is one loop over that table. After each move the program redraws that cell, runs check_win for the current player, checks move_count for a draw at 9, and switches players with $s0 = 3 - $s0. Empty cells show a dim number so you know what to type, X is two thick diagonals, O is a ring, and when someone wins highlight_line redraws the three winning marks in green and the strip at the bottom says X WINS!, O WINS! or DRAW.

Procedures: main (game loop, prompt and validation, play again), reset_game, check_win(player), highlight_line(line, player), print_console_board and print_cell, show_turn_status and show_status, draw_board_screen, cell_origin(index) which uses div/mflo/mfhi, draw_mark, draw_mark_color, draw_x_mark, draw_o_mark, plus read_line, parse_int and the graphics procedures.

T1: X wins down the middle column

T2: O wins on the diagonal

T3: full board, draw

Tests

- **T1** X wins with the middle column. Typed: 5 1 9 3 2 7 4 6 8, N. Passed.
- **T2** O wins on the anti-diagonal. Typed: 1 3 2 5 4 7, N. Passed.
- **T3** Full board with no winner (draw). Typed: 1 2 3 5 4 6 8 7 9, N. Passed.
- **T4** (edge case) Invalid moves: text, empty line, 0, 10, spaces, occupied cell. Typed: abc, (empty), 0, 10, ' 5 ', 5 (taken), 1, 9, 3, 2, 7, 4, 6, 8, N. Passed.
- **T5** (edge case) Play-again validation and board reset for a second game. Typed: 1 4 2 5 3, maybe, y, 1 2 3 5 4 6 8 7 9, n. Passed.

T4 log, bad input then a taken cell (what I typed is after each prompt)

```
Player X, choose a cell (1-9): abc
  ! That is not a whole number. Enter a digit from 1 to 9.
Player X, choose a cell (1-9):
  ! That is not a whole number. Enter a digit from 1 to 9.
Player X, choose a cell (1-9): 0
  ! Out of range. Enter a digit from 1 to 9.
Player X, choose a cell (1-9): 10
  ! Out of range. Enter a digit from 1 to 9.
Player X, choose a cell (1-9):  5

 1 | 2 | 3
---+---+---
 4 | X | 6
---+---+---
   ...
```

T3 log, end of the draw game

```
Player X, choose a cell (1-9): 9

 X | O | X
---+---+---
 X | O | O
---+---+---
 O | X | X


*** It's a draw. ***
Play again? (Y/N): N
Thanks for playing!
```

Game 2: Blackjack

You play against a dealer that hits on 16 or less and stands on 17 or more. The deck is a 52 byte array holding 0 to 51, where rank is card mod 13 and suit is card / 13. Every round refills and shuffles it with a Fisher-Yates loop using syscall 42, seeded once from the clock (syscalls 30 and 40). deal_card appends the next card to a hand array and bumps its counter. hand_value counts aces as 11 and then subtracts 10 per ace while the total is over 21, so A+6 is 17 and A+6+9 is 16. A natural blackjack is checked first, 21 stands automatically, a bust ends the round, otherwise the dealer flips the hidden card and draws to 17. Wins, losses and pushes are counted across rounds. Cards on the display are 9 by 12 with the rank and a suit letter, red for hearts and diamonds, and the dealer's second card is a blue card back until you stand.

Procedures: main, shuffle_deck, deal_card(hand, count pointer), card_value, hand_value, to_upper, print_card, print_hand, print_player_line, print_table_hidden, print_dealer_full, redraw_table(hide dealer), draw_hand, draw_card, rank_char, draw_scores, draw_record, draw_number and int_to_str (pushes digits on the stack to reverse them).

My turn, dealer card hidden

Round over, 19 vs 18, I won

After the 40 round run

Tests

- **B1** Stand immediately; dealer plays out its hand. Typed: S, N. Passed.
- **B2** Basic strategy: hit under 17, stand at 17+. Typed: H while total < 17, then S; N. Passed.
- **B3** Three rounds in a row; win/loss/push counters accumulate. Typed: basic strategy, Y, Y, N. Passed.
- **B4** (edge case) Invalid commands (7, empty, ?) are rejected without dealing. Typed: 7, (empty), ?, S, N. Passed.
- **B5** (edge case) Hit until bust (or 21), then invalid play-again answer. Typed: H repeatedly, maybe, N. Passed.

The cards are random so every run is different. I also ran 40 rounds with a script that hits under 17 (B6) and a checker recomputed every printed hand total from the card names and checked the dealer's 17 rule and the announced result. 113 totals over 40 rounds, no mistakes, final record 18 won 19 lost 3 pushed.

B3 log, one round of the basic strategy run

```
---------------- NEW ROUND ----------------
Dealer shows : AC  [??]
Your hand    : 8D 9H    (total 17)
(H)it or (S)tand? S
Dealer reveals: AC 6H
Dealer hand  : AC 6H    (total 17)

*** Push - it's a tie. ***
Record: 0 won, 0 lost, 1 pushed.
Play another round? (Y/N): Y

---------------- NEW ROUND ----------------
   ...
```

B4 log, bad commands are rejected

```
---------------- NEW ROUND ----------------
Dealer shows : JS  [??]
Your hand    : 8H 6D    (total 14)
(H)it or (S)tand? 7
  ! Please type H to hit or S to stand.
(H)it or (S)tand?
  ! Please type H to hit or S to stand.
(H)it or (S)tand? ?
  ! Please type H to hit or S to stand.
(H)it or (S)tand? S
Dealer reveals: JS QS
Dealer hand  : JS QS    (total 20)

*** Dealer wins. ***
   ...
```

Game 3: Hangman (my custom game)

I picked Hangman because it uses the requirements differently from the other two: a table of pointers to strings for the word list, a 26 entry array indexed by letter minus 'A' for the letters tried, character checking on input, and a drawing that grows one piece at a time. The word is picked with syscall 42 from twelve computer words (COMPUTER, ASSEMBLY, REGISTER, PROCEDURE, MEMORY, BRANCH, STACK, SYSCALL, PIPELINE, COMPILER, HARDWARE, PROGRAM). A guess is accepted only if it is exactly one character, a letter, and not tried yet. A repeat does not cost a life. count_in_word gives how many times the letter appears, word_solved checks the word against the guessed array, and six misses loses, which reveals the word with the missing letters in red. draw_figure(n) draws the first n parts of the person (head, body, arms, legs) from the wrong guess counter, so the scene is rebuilt from the state every redraw.

Procedures: main, reset_game, count_in_word(letter), word_solved, to_upper, print_state, draw_scene, draw_lives, draw_gallows, draw_figure(parts), draw_diag(x, y, dx), draw_word_row(reveal all), draw_miss_row, draw_number, int_to_str.

H1: solved, no misses

H2: solved after three misses

H3: six misses, word revealed

Tests

- **H1** Solve the word with no misses (word-aware guesser). Typed: letters chosen from the word list; N. Passed.
- **H2** Win after three misses (figure partly drawn). Typed: Z Q J, then word-aware letters; N. Passed.
- **H3** Six misses: the game is lost and the word is revealed. Typed: F J Q V X Z, N. Passed.

- **H4** (edge case) Invalid guesses: two letters, empty, digit, symbol, repeat (no life lost). Typed: ab, (empty), 1, ?, e, E (repeat), F J Q V X Z, N. Passed.
- **H5** (edge case) Lower-case input accepted; play-again validation; new word and 6 lives on replay. Typed: f j q v x z, xyz, Y, F J Q V X Z, n. Passed.

The word is random, so for H1 and H2 a small script picked the letter that appears in the most words still matching the pattern. The losing tests use F, J, Q, V, X and Z because none of the twelve words have those letters.

H4 log, bad guesses and a repeat

```
Guess a letter: ab
  ! Enter exactly one letter.
Guess a letter:
  ! Enter exactly one letter.
Guess a letter: 1
  ! That is not a letter (A-Z only).
Guess a letter: ?
  ! That is not a letter (A-Z only).
Guess a letter: e
  Yes! 'E' appears 1 time(s).
Word : _ _ _ _ _ _ E _    Lives: 6   Missed:
Guess a letter: E
  ! You already tried that letter.
Guess a letter: F
   ...
```

H2 log, end of the game

```
*** You got it! The word was SYSCALL. ***
Score: 1 won, 0 lost.
Play again? (Y/N): N
Thanks for playing!
```

Test summary

Every test ran on the MIPS program in MARS and on the Java program with the same input. 31 of 31 passed. Edge cases covered: letters and blank lines where a number goes, numbers out of range, taken cells, bad commands, multi character and non letter guesses, repeated guesses, bad play again answers, state resetting between games, soft aces, natural blackjack, the dealer's 17 rule, and a bust ending the round before the dealer plays.

Test	Game	MIPS	Java
T1 X wins with the middle column	tictactoe	pass	pass
T2 O wins on the anti-diagonal	tictactoe	pass	pass
T3 Full board with no winner (draw)	tictactoe	pass	pass
T4 Invalid moves: text, empty line, 0, 10, spaces, occupied cell	tictactoe	pass	pass
T5 Play-again validation and board reset for a second game	tictactoe	pass	pass
B1 Stand immediately; dealer plays out its hand	blackjack	pass	pass
B2 Basic strategy: hit under 17, stand at 17+	blackjack	pass	pass
B3 Three rounds in a row; win/loss/push counters accumulate	blackjack	pass	pass
B4 Invalid commands (7, empty, ?) are rejected without dealing	blackjack	pass	pass
B5 Hit until bust (or 21), then invalid play-again answer	blackjack	pass	pass
H1 Solve the word with no misses (word-aware guesser)	hangman	pass	pass
H2 Win after three misses (figure partly drawn)	hangman	pass	pass
H3 Six misses: the game is lost and the word is revealed	hangman	pass	pass

Test	Game	MIPS	Java
H4 Invalid guesses: two letters, empty, digit, symbol, repeat (no life lost)	hangman	pass	pass
H5 Lower-case input accepted; play-again validation; new word and 6 lives on replay	hangman	pass	pass
B6 40 round blackjack check	blackjack	pass	-

Source code

All of the code is on GitHub at github.com/dflo042/cs in the mips-game-project folder. mips/ has tictactoe.asm, blackjack.asm and hangman.asm, java/ has the Java planning versions (TicTacToe.java, Blackjack.java, Hangman.java), and display_renders/ has the display images from my test runs. Each .asm file starts with a comment block that says how to run it, what the data structures are, and the register conventions.

I wrote the Java versions first to work out the game logic and the display layout. Each static method became a MIPS procedure with the same name, the arrays and counters match the .data section, and the Display class in each Java file is a model of the Bitmap Display with the same font and drawing routines, so porting was mostly turning methods into procedures and ifs and whiles into branches and jumps.

Sources and acknowledgments

- ZyBooks Chapter 4 for procedures, the stack, arrays and branching.
- The MARS help menu for the syscall numbers (4, 5, 8, 10, 11, 30, 40, 42) and the Bitmap Display settings. MARS 4.5 is by Pete Sanderson and Ken Vollmar (dpetersanderson.github.io).
- Fisher-Yates shuffle and the dealer's hit on 16 stand on 17 rule are the standard way to do those, no code was copied.
- For testing I ran the programs in MARS from the command line with scripted input and used Python scripts to type the answers, dump the display memory to PNG files and double check the blackjack math. Java was OpenJDK 21.
- I used Claude, an AI assistant, to help write and debug the MIPS and Java code, the test scripts and this write up. I went through the code, ran all the tests, and I am responsible for the submission.
- I did not work with other students on this.